

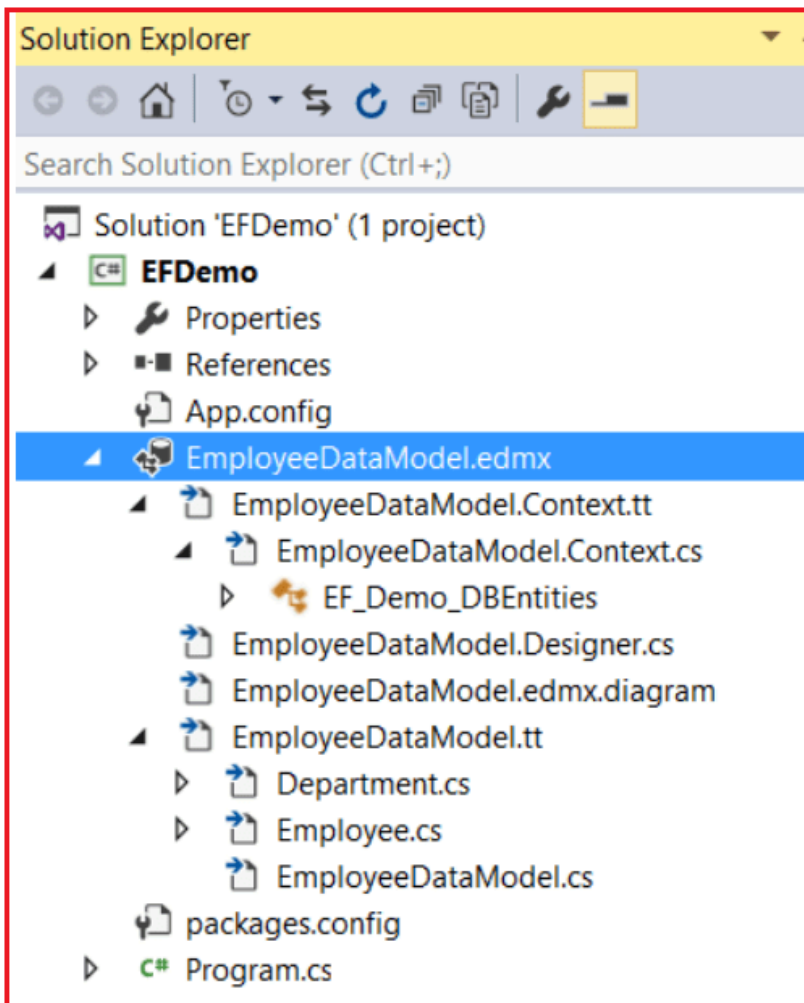
Entities in Entity Framework in C# with Examples

In this article, I am going to discuss the **Entities in the Entity Framework in C#** with Examples. Please read our previous article where we discussed [Entity Framework Context Class](#) in C# with Examples. At the end of this article, you will understand the following pointers in detail with Examples.

1. [What is an Entity in the Entity Framework?](#)
2. [Understanding Scalar Property and Navigation Property](#)

What is an Entity in the Entity Framework?

We are going to work with the same example, that we worked on in our [Previous](#) articles. Let's see the Solution of the console application that we created in the [Introduction to Entity Framework](#) article of this article series.



Now, if you go to the context class, you will see that the context class includes two properties, one for each entity as shown in the below image.

```

public partial class EF_Demo_DBEntities : DbContext
{
    public EF_Demo_DBEntities()
        : base("name=EF_Demo_DBEntities")
    {
    }

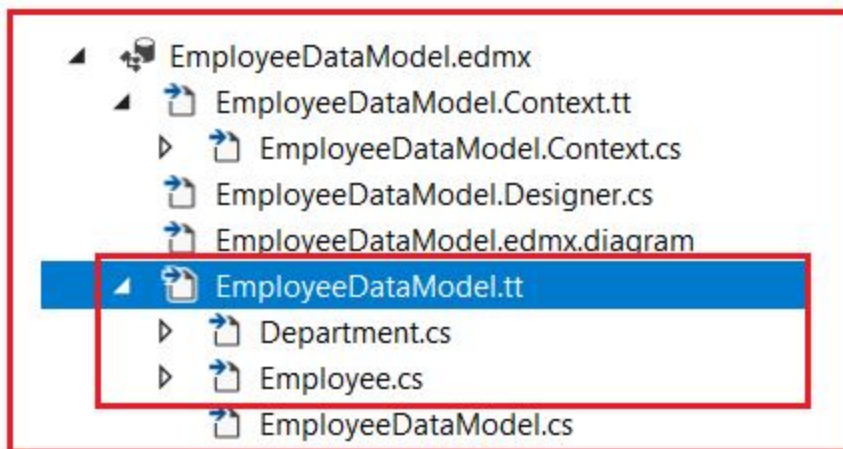
    protected override void OnModelCreating(DbModelBuilder modelBuilder)
    {
        throw new UnintentionalCodeFirstException();
    }

    public virtual DbSet<Department> Departments { get; set; }
    public virtual DbSet<Employee> Employees { get; set; }
}

```

 **Entities**

As you can see in the above image, Departments property type is `DbSet<Department>` and Employees property type is `DbSet<Employee>`. So, here, Department and Employee are nothing but entities. An Entity in Entity Framework is a class that is included as a **`DbSet<TEntity>`** type property in the derived context class. Entity Framework maps each entity to a database table and each property of an entity is mapped to a column in the database table. In our example, the Department entity is mapped with the Departments database table and the Employee entity is mapped with the Employees database table. These two entities are generated automatically by Entity Framework and you can find these two entities inside the EmployeeDataModel.tt file as shown in the below image.



Department Entity:

The Department Entity or Employee domain class is created with the following definition. As the Departments table contains three columns, so entity framework created the following Department Entity with three properties to hold the ID, Name, and Location values. Further, if you remember we have created the Employees table by creating a foreign key referring to the ID column of the Departments table. So, that means, a particular department may have multiple employees. As we have made the foreign key relationship, to hold the list of employees of a particular department, it also includes a property i.e. Employees. And through the constructor of the class, it is initializing the Employees ICollection property.

```
//-----
---
// <auto-generated>
//     This code was generated from a template.
//
//     Manual changes to this file may cause unexpected behavior in your
// application.
//     Manual changes to this file will be overwritten if the code is
// regenerated.
// </auto-generated>
//-----
---
namespace EFDemo
{
    using System;
    using System.Collections.Generic;

    public partial class Department
    {
        [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage",
"CA2214:DoNotCallOverridableMethodsInConstructors")]
        public Department()
        {
            this.Employees = new HashSet<Employee>();
        }

        public int ID { get; set; }
        public string Name { get; set; }
        public string Location { get; set; }

        [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage",
"CA2227:CollectionPropertiesShouldBeReadOnly")]
        public virtual ICollection<Employee> Employees { get; set; }
    }
}
```

Employee Entity:

The Employee Entity or Employee domain class is created with the following definition. As the Employees table contains six columns, so the Entity Framework created the following Employee Entity with six properties to hold the ID, Name, Email, Gender, Salary, and DepartmentId values. Further, if you remember we have created the Employees table by creating a foreign key referring to the ID column of the Departments table. That means one employee belongs to a particular department. To store that department information, it includes a property called Department which is of type Department entity.

```
//-----
---
// <auto-generated>
//     This code was generated from a template.
//
```

```
//      Manual changes to this file may cause unexpected behavior in your
//      application.
//      Manual changes to this file will be overwritten if the code is
//      regenerated.
// </auto-generated>
//-----
---
namespace EFDemo
{
    using System;
    using System.Collections.Generic;

    public partial class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Email { get; set; }
        public string Gender { get; set; }
        public Nullable<int> Salary { get; set; }
        public Nullable<int> DepartmentId { get; set; }

        public virtual Department Department { get; set; }
    }
}
```

The point that you need to remember above Department and Employee classes become entities when they have included as **DbSet<TEntity>** properties in the context class (the class which is derived from DbContext class). As we have already seen, the context class included these classes as **DbSet<Department > and DbSet<Employee >**.

The Department and Employee are entities. An Entity can include two types of properties: Scalar Properties and Navigation Properties. Let us proceed further and understand what do you mean by Scalar and Navigation Properties in Entity Framework.

Scalar Property in C#:

The Primitive Type Properties defined inside a class are called Scalar Properties. Scalar property stores the actual data. A scalar property maps to a single column in the database table. For example, in the Employee class below are the scalar properties

```
public int ID { get; set; }
public string Name { get; set; }
public string Email { get; set; }
public string Gender { get; set; }
public Nullable<int> Salary { get; set; }
public Nullable<int> DepartmentId { get; set; }
```

Similarly, in the Department Class Below are the Scalar properties

```
public int ID { get; set; }
public string Name { get; set; }
```

```
public string Location { get; set; }
```

Navigation Property in C#:

The Navigation Property represents a relationship with another Entity. There are two types of navigation properties. They are as follows:

1. [Reference Navigation Property](#)
2. [Collection Navigation Property](#)

Reference Navigation Property in C#:

If an entity includes a property of another entity type, then it is called a Reference Navigation Property in C#. It represents the multiplicity of one (1) i.e. it represents the one-to-one relationship between the entities. For example, in Employee Class, the following property is a Reference Navigation property. This indicates one Employee belongs to one Department.

```
public virtual Department Department { get; set; }
```

Collection Navigation Property in C#:

If an entity includes a property of collection type, it is called a Collection Navigation Property in C#. It represents the multiplicity of many (*) i.e. represents one-to-many relationships. For example, in the Department class following property is a collection navigation property. This indicates that one Department has many employees.

```
public virtual ICollection<Employee> Employees { get; set; }
```

In the next article, I am going to discuss the [Entity Types in the Entity Framework](#) with Examples. Here, in this article, I try to explain **Entities in the Entity Framework in C# with Examples**. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about Entities in the Entity Framework article.